



BugIt
by Taskivator

Manual completo paso a paso

BugIt

Agente de QA para registrar bugs en VS Code

Configuración inicial y uso diario: una guía detallada, clic a clic, escrita para personas que nunca han hecho algo así antes. No se necesitan conocimientos técnicos. Sigue el orden y hoy mismo estarás registrando tickets de bug limpios.

Se ejecuta en: VS Code en Windows (principal), macOS & Linux

Aviso sobre la traducción.

Este documento se ha traducido automáticamente y no ha sido revisado por hablantes nativos. La versión en inglés es la que prevalece: en caso de discrepancia, rige el texto en inglés. Para consultar la redacción más precisa y actualizada, acude al documento en inglés.

CONTENIDO

Haz la **Parte 1** una vez, en orden. Todo lo demás puedes consultarlo cuando lo necesites. La **Solución de problemas** y las **Preguntas frecuentes** cerca del final responden las dudas más comunes, así que revísalas antes de escribirnos a soporte.

Parte 1 · Configuración (hazlo una vez)

- Paso 0: Consigue GitHub Copilot
- Paso 1: Instala VS Code
- Paso 2: Descomprime tu descarga de BugIt
- Paso 3: Abre BugIt en VS Code
- Paso 4: Elige el agente BugIt en el chat
- Paso 5: Activa tu licencia
- Paso 6: Acepta los términos (una vez)

Paso 7: Ejecuta "Begin setup"

Paso 8: Conecta tu sistema de seguimiento

Paso 9: Tus inicios de sesión guardados

Paso 10: Confirma que estás listo

Parte 2 · Uso diario

Comandos de un vistazo

Parte 3 · Hazlo tuyo

Parte 4 · Actualizaciones, copias de seguridad y seguridad

Parte 5 · Solución de problemas y preguntas frecuentes

Preguntas frecuentes

Licencias de Equipo: preguntas

Parte 6 · Licencia y soporte

La única regla que te mantiene a salvo

BugIt **nunca** registra nada sin antes mostrarte una vista previa y pedirte una confirmación escrita: para las acciones irreversibles debes escribir **FILE IT**, y un simple «sí» no basta. Siempre tienes el control. ¿Quieres practicar sin ningún riesgo? Escribe **dry run** y escribirá el reporte completo **sin** registrarlo: el dry-run suprime todo ticket, comentario, edición, adjunto, eliminación y notificación externos (los comandos locales que ejecutas a propósito, como copia de seguridad o exportación, todavía pueden crear archivos locales) e indica al agente que rechace las escrituras en el sistema de seguimiento (ese rechazo sigue las instrucciones de BugIt, no es un bloqueo de la plataforma; usa credenciales de solo lectura para evaluar).

Configuración inicial frente a uso diario

Solo sigues toda la **Parte 1** una vez. Después de eso, abrir BugIt es rápido, y las actualizaciones son un solo comando (**Update**). Consulta la Parte 4.

Unos 15 minutos. Sigue los pasos en orden; cada uno se apoya en el anterior. No te saltes ninguno.

0 Consigue GitHub Copilot

Esta guía sigue la ruta de GitHub Copilot

Buglt es el "conductor" y la IA es el "motor" (es la que realmente escribe los reportes). Estos pasos lo configuran con GitHub Copilot en VS Code, la ruta más rápida si empiezas desde cero. Buglt también funciona con la extensión de Claude, con cualquier otro asistente que pueda ejecutar un comando, y por su cuenta con tu propia clave de IA. Si ya usas alguno de ellos, salta al Paso 2 y abre la carpeta de Buglt en esa herramienta en lugar de VS Code.

Qué es: GitHub Copilot es un asistente de IA de GitHub (propiedad de Microsoft). Tiene un plan gratuito para uso ligero y un plan de pago para uso intensivo. Lo instalarás en el Paso 1 e iniciarás sesión.

1. Necesitarás una **cuenta de GitHub**. Si no tienes una, crea una gratis en github.com/signup.
2. El plan gratuito de Copilot es suficiente para probar Buglt. Si registras bugs todo el día, vale la pena el plan de pago "Copilot Pro", y puedes actualizar cuando quieras en github.com/features/copilot.
3. Activarás Copilot dentro de VS Code en el Paso 1, así que no necesitas hacer nada más aquí todavía.

1 Instala VS Code

VS Code es una app gratuita de Microsoft. Es la "ventana" en la que vive Buglt.

1. Abre tu navegador web y ve a code.visualstudio.com.
2. Haz clic en el gran botón azul **Download** (detecta tu sistema automáticamente).
3. Abre el archivo descargado e instálalo: **acepta todas las opciones predeterminadas**, solo sigue haciendo clic en Next/Install.
4. Abre VS Code una vez instalado.

Activa GitHub Copilot dentro de VS Code

1. En el extremo izquierdo de VS Code, haz clic en el icono **Extensions** (parece cuatro cuadrados pequeños).
2. En el cuadro de búsqueda, escribe **GitHub Copilot**.
3. Haz clic en **Install** en "GitHub Copilot" (y, si se ofrece, en "GitHub Copilot Chat").
4. Un aviso te pedirá **Sign in to GitHub**. Haz clic en él e inicia sesión con tu cuenta de GitHub en la ventana del navegador que se abre.

✓ **Sabrás que funcionó cuando:** aparezca un pequeño icono de Copilot en la parte inferior de VS Code, y se abra el panel de Chat (el icono de burbuja de diálogo a la izquierda, o pulsa **Ctrl+Alt+I**).

2 Descomprime tu descarga de Buglt

Tu compra incluyó un archivo .zip (por ejemplo `buglt-qa-agent.zip`). Necesitas descomprimirlo; ejecutarlo comprimido no funcionará.

1. Busca el archivo .zip (normalmente en tu carpeta **Downloads**).
2. Haz **clic derecho** en él → elige **Extract All...** (Windows) o haz doble clic en él (Mac).
3. Elige una ubicación sencilla como tu carpeta **Documents**, luego haz clic en **Extract**.
4. Ahora tendrás una carpeta **Buglt** con muchos archivos dentro. Recuerda dónde está.

No la pongas en OneDrive ni en una carpeta sincronizada

Las carpetas sincronizadas en la nube pueden causar conflictos de archivos. Tu carpeta Documents o el Escritorio son perfectos. Tampoco renombres los archivos de dentro; deja la carpeta tal cual.

3 Abre Buglt en VS Code

1. En VS Code, haz clic en **File → Open Folder** (menú superior izquierdo).
2. Ve hasta donde descomprimiste Buglt (p. ej., Documents).
3. Haz clic una vez en la carpeta **Buglt** para seleccionarla, luego haz clic en **Select Folder**.
4. Si VS Code pregunta "Do you trust the authors of the files in this folder?", haz clic en **Yes, I trust the authors**.

✓ **Sabrás que funcionó cuando:** veas los archivos de Buglt (carpetas como `tools`, `.github`, y archivos como `README.md`) listados en el lado izquierdo de VS Code.

4 Elige el agente Buglt en el chat

1. Abre el panel de **Chat** de Copilot: haz clic en el icono de burbuja de diálogo a la izquierda, o pulsa `Ctrl+Alt+I` (Windows) / `Cmd+Alt+I` (Mac).
2. En la parte superior del cuadro de chat hay un pequeño menú desplegable de modo (puede decir "Ask" o "Agent").
3. Haz clic en él y elige **Buglt QA Agent** en la lista.

¿No ves "Buglt QA Agent" en la lista?

Asegúrate de haber abierto la carpeta Buglt (Paso 3), no otra carpeta; el agente se carga desde dentro de ella. Cierra y vuelve a abrir la carpeta si es necesario.

5 Activa tu licencia

Una cuenta de Buglt es todo lo que necesitas; no hay ninguna clave de licencia que copiar ni pegar. Apruebas este dispositivo en tu navegador, y el comando `Activate` termina el resto.

1. En el cuadro de chat, escribe `hi` y pulsa Enter.
2. Escribe `Activate`. Buglt abre el Buglt Portal en tu navegador. Inicia sesión en tu propia cuenta, elige tu titularidad Solo o Team y aprueba este dispositivo. Vuelve a VS Code y Buglt termina de autorizar automáticamente; tu contraseña permanece en el navegador y nunca se introduce en VS Code.

✓ **Sabrás que funcionó cuando:** Buglt confirme la activación y nombre la cuenta con la que has iniciado sesión.

Mantén tu cuenta en privado

Tu cuenta y tu titularidad están vinculadas a ti. No las compartas, publiques ni revendas; un acceso compartido puede revocarse. Cada miembro del equipo (Team) inicia sesión con su propia cuenta; no hay ninguna clave compartida ni inicio de sesión compartido. Cambiar a otra computadora más adelante no hay problema (consulta las Preguntas frecuentes).

6 Acepta los términos (una vez)

La primera vez, Buglt muestra un breve resumen y te pide que lo aceptes antes de hacer cualquier otra cosa.

1. Lee el breve resumen que muestra (tú revisas cada ticket antes de que se registre; la seguridad de tu proyecto es tu responsabilidad; las salvaguardas son ayudas, no garantías, y como las sigue el modelo de IA que te esté respondiendo en Copilot, un modelo más ligero/gratuito puede a veces saltarse un paso o necesitar que repitas un comando, algo que un modelo más potente no haría).
2. Elige **I agree** entre las dos opciones que muestra Buglt.

✓ **Sabrás que funcionó cuando:** Buglt confirme y continúe con la configuración. Solo se te pregunta esto una vez; lo recuerda.

7 Ejecuta "Begin setup"

Ahora Buglt se configura solo entrevistándote en lenguaje sencillo. Nunca editas un archivo de ajustes a mano.

ESCRIBES

Begin setup

Te hará preguntas breves y activará solo lo que elijas. La más importante es tu sistema de seguimiento.

8 Conecta tu sistema de seguimiento

La configuración registró *qué* sistema de seguimiento usas. Este paso es el que permite que Buglt registre tickets en él de verdad: creas un token de API en tu propia cuenta y lo guardas en esta computadora con un solo comando. Buglt registra los tickets directamente a través de la API REST de tu sistema de seguimiento con ese token, así que no hace falta dejar nada en ejecución y el registro nunca usa un servidor MCP.

8a · Crea un token en tu propia cuenta

TU SISTEMA DE SEGUIMIENTO	DÓNDE CREAMLA
Jira Cloud	id.atlassian.com/manage-profile/security/api-tokens
Azure DevOps	dev.azure.com/YOUR-ORG/_usersSettings/tokens
GitHub Issues	github.com/settings/personal-access-tokens/new
GitLab Issues	gitlab.com/-/user_settings/personal_access_tokens
Bugzilla	en tu propio Bugzilla: Preferences → API Keys
YouTrack	en tu propio YouTrack: avatar → Profile → Account Security → Authentication
Linear	linear.app/settings/account/security
Shortcut	app.shortcut.com/settings/account/api-tokens
ClickUp	app.clickup.com/settings/apps
Asana	app.asana.com/0/my-apps
Trello	trello.com/apps/admin (una clave de API Y un token que autorizas con ella)

Copia tu token en cuanto aparezca

La mayoría de los servicios muestran un token nuevo solo una vez. Cópialo de inmediato y guárdalo en un lugar seguro hasta que Buglt lo guarde por ti (Paso 9). Elige la caducidad más larga que ofrezca tu sistema de seguimiento para no tener que repetir esto pronto.

8b · Guárdalo en esta computadora

Pide a tu asistente que conecte tu sistema de seguimiento y lo hará por ti. O, abre un terminal dentro de VS Code y ejecuta el comando de tu sistema de seguimiento, usando `ado`, `github`, `gitlab` y demás en lugar de `jira`.

EJECUTAS (EN EL TERMINAL)

```
python tools/connect.py jira
```

8c · Pega el token en el mensaje enmascarado

El comando pide el token en un mensaje local enmascarado: nunca se muestra en pantalla y nunca llega al chat, a un ticket ni a `config.json`. Pégallo ahí y en ningún otro sitio. Buglt inicia sesión, enumera los proyectos que el token ve realmente, lee los tipos de incidencia que puedes crear y **se niega a guardar una credencial que no pueda registrar tickets**. Por eso, si la configuración termina, el registro funciona.

8d · Comprueba que está conectado

Para confirmarlo en cualquier momento, ejecuta la comprobación en el terminal o pregunta en el chat:

EJECUTAS (EN EL TERMINAL)

```
python tools/connect.py jira --check
```

ESCRIBES

```
Check status
```

✓ **Sabrás que funcionó cuando:** Buglt indique que tu sistema de seguimiento está conectado / en buen estado.

No hace falta dejar nada en ejecución

Tu token de API vive en el almacén de credenciales de tu sistema operativo, no en VS Code. No hay nada que iniciar cuando vuelvas a abrir VS Code: Buglt puede registrar de inmediato. Puedes comprobarlo cuando quieras con `python tools/connect.py jira --check`.

9 Tus inicios de sesión guardados

Cuando guardas tu token (Paso 8c), tu propia computadora las protege: en Windows como un registro cifrado que solo tu cuenta de Windows en esa máquina puede abrir, y en macOS y Linux en el Llavero o en el servicio de secretos. Nunca se escriben en un archivo de texto plano, y no se envían a ningún sitio salvo al sistema de seguimiento que configuraste. No tienes que hacer nada más: la próxima vez que abras VS Code, tus datos guardados ya están ahí.

¿Quieres confirmar qué sistemas de seguimiento ya tienen una credencial guardada? Solo escribe:

ESCRIBES

```
Check saved credentials
```

...y Buglt te muestra qué sistemas de seguimiento tienen una credencial guardada; nunca los valores de las credenciales. Buglt nunca muestra ni copia el valor de un token guardado.

10 Confirma que estás listo

Pídele a Buglt que verifique todo dos veces antes de registrar algo de verdad:

ESCRIBES

Check readiness

Enumera todo lo que aún falte (normalmente solo "conecta tu sistema de seguimiento" o "activa"). Cuando todo esté en verde, verás "✅ Setup complete."

Eso es todo: la configuración está hecha para siempre

No repetirás la Parte 1. A partir de ahora, abrir Buglt es: abrir la carpeta → escribir `hi`. Para cambiar cualquier elección más adelante, solo escribe `Begin setup` y di qué quieres cambiar.

¿Sin GitHub Copilot? Hay un asistente de terminal

Si no puedes usar Copilot, abre la Terminal integrada (Terminal → New Terminal) y ejecuta `python tools/setup_wizard.py`. Hace unas pocas preguntas y escribe tus ajustes sin la IA. También puedes ejecutar Buglt de forma independiente con tu propia clave de IA; consulta las Preguntas frecuentes.

PARTE 2 · USO DIARIO

Una vez configurado, trabajas completamente desde el chat. Habla con naturalidad, o usa los ejemplos exactos de abajo. Escribe `help` en cualquier momento para ver la lista completa.

Escribir un reporte de bug

El trabajo principal de Buglt. Describe el bug en una frase sencilla; escribe el ticket completo, autocompleta la versión y la categoría, comprueba si hay duplicados, te muestra una vista previa y lo registra después de que escribas `FILE IT` para confirmar (un simple «sí» no basta).

ESCRIBES

Write a bug report: la app se cierra cada vez que toco Guardar en iPhone

Redacta un título, pasos para reproducirlo, resultado esperado frente al real, severidad y plataforma, y registra en el texto del ticket la severidad que confirmes. Cuando tu sistema de seguimiento y tu proyecto ofrecen un campo nativo de prioridad o severidad, Buglt también lo establece. GitHub, GitLab y Trello no tienen ese campo. ¿Quieres cambios antes de registrarlo? Solo dilo, por ejemplo "pon la severidad en alta" o "añade que empezó en la última versión."

Bugs rápidos (registro veloz)

Para los casos comunes (crash, error, missing, layout, slow, stuck, data, visual, audio, blocker), el formulario rápido se salta algunas preguntas y completa la categoría y la prioridad por ti, y omite la búsqueda de duplicados para ahorrar el viaje de ida y vuelta. Úsalo cuando ya sepas que el problema es nuevo; la vista previa lo indica, y un informe estándar sigue ejecutando la comprobación completa de duplicados.

ESCRIBES (EJEMPLOS)

Quick bug: crash al iniciar tras la actualización

Quick bug: layout el botón se superpone con el texto en la pantalla de ajustes

Quick bug: blocker no puedo continuar después del paso de pago

Quick bug: slow el panel tarda 20 segundos en cargar

Reportes de bugs de fallos (crash)

Si conectaste una herramienta de fallos (como Sentry), BugIt hace todo lo anterior además de relacionar tu reporte con el fallo y obtener el stack trace por ti.

ESCRIBES

Write a crash bug report: el juego se cierra al abrir el mapa

Comentarios de verificación y cierre

Después de probar una corrección, BugIt escribe un comentario ordenado para publicar en el ticket. Escribe (y opcionalmente publica) el comentario; no cambia el estado del ticket. Tú sigues cerrando/resolviendo/reabriendo el ticket en tu sistema de seguimiento.

ESCRIBES	QUÉ OBTIENES
Close #123	Pregunta qué escenario (corrección confirmada, cerrado a petición, funciona como se diseñó, reabrir...) y luego escribe ese comentario.
Escribe un comentario de verificación para la compilación que probaste en Windows	Un comentario de "corrección confirmada" con los detalles de la compilación completados.
Escribe un comentario de reapertura para la compilación que probaste en Windows	Un comentario de "el problema sigue ocurriendo" (luego reabres el ticket).

Traducción

Traduce reportes o términos entre tus idiomas configurados, usando la redacción exacta de tu equipo (de tu glosario; consulta la Parte 3).

ESCRIBES (EJEMPLOS)

Translate to ja: el botón de guardar no hace nada en la pantalla de perfil

Traduce este reporte de bug al inglés: [pega el texto]

Busca información (especificaciones y glosario)

Si conectaste especificaciones o completaste tu glosario, hazle preguntas a BugIt sobre tu proyecto.

ESCRIBES (EJEMPLOS)

¿Qué es [tu término]?

Explicame el flujo de compra

¿Qué hay de nuevo en las especificaciones?

Detección de duplicados

Esto ocurre automáticamente en un informe estándar, antes de que se escriba nada: BugIt busca en tu sistema de seguimiento tickets coincidentes (incluso los redactados de forma un poco distinta) y te avisa antes de que se escriba nada, para que decidas tú. Es una comprobación y no una garantía: la búsqueda lee un número limitado de tickets, así que un duplicado que no encuentre sigue pudiendo registrarse. El formulario rápido omite la búsqueda a propósito para ahorrar el viaje de ida y vuelta, y lo indica en la vista previa, así que siempre sabes cuál de los dos has recibido.

Funciones avanzadas (si las activaste)

ESCRIBE ESTO	QUÉ HACE
<code>Triage digest</code>	Standup instantáneo: bugs abiertos por área/severidad, los más antiguos señalados.
<code>Export bug to file</code>	Guarda el reporte como Markdown/CSV/JSON en lugar de registrarlo (ideal antes de tener el sistema de seguimiento configurado).
<code>Undo last filing</code>	Muestra el último registro del historial de auditoría. BugIt no puede eliminar ni cerrar un ticket; tras una nueva vista previa y <code>FILE IT</code> solo puede eliminar un comentario o un adjunto creados por él mismo.
(automático)	Alertas de SLA en bugs urgentes · campos específicos por categoría.

La lista completa de comandos

ESCRIBES

`help`

...muestra todos los comandos dentro del agente, en cualquier momento.

Practica de forma segura en cualquier momento

Escribe `dry run` y BugIt escribe todo el reporte **sin** registrar nada, perfecto para aprender o probar. El `dry-run` suprime todo ticket, comentario, edición, adjunto, eliminación y notificación externos (los comandos locales que ejecutas a propósito, como copia de seguridad o exportación, todavía pueden crear archivos locales) e indica al agente que rechaza las escrituras en el sistema de seguimiento; ese rechazo sigue las instrucciones de BugIt, no es un bloqueo de la plataforma, así que usa credenciales de solo lectura si quieres evaluarlo con total garantía.

Comandos de un vistazo

Siempre puedes describir lo que quieres con tus propias palabras. Estas son las frases exactas que BugIt reconoce, por si las prefieres.

DI ESTO	QUÉ OCURRE
Registrar un bug	
<code>Quick bug: [type] [subject]</code>	Registrar rápido. Se salta la búsqueda de duplicados a propósito y lo indica en la vista previa.
<code>dry run</code>	Escribir el informe completo y no registrar nada. La forma segura de practicar.
<code>FILE IT</code>	Lo único que registra. Escrito como tu mensaje completo, una vez, para el borrador que acabas de leer.
<code>Export bug to file</code>	Guardar el informe como archivo en lugar de enviarlo a tu sistema de seguimiento.
<code>Undo last filing</code>	Muestra la última escritura y puede quitar un comentario o adjunto que añadió BugIt. No puede cerrar un ticket. Requiere el registro de auditoría activado.
<code>Triage digest</code>	Un resumen de lo que has registrado últimamente.
El conocimiento de tu proyecto	
<code>What is [term]?</code>	Buscar un término en tu glosario.
<code>What's new in specs?</code>	Cambios recientes en la carpeta de especificaciones que indicaste a BugIt.
<code>Look up crash [ID]</code>	Buscar en tu herramienta de fallos y devolver solo enlaces. No se escribe ningún informe.
<code>Look up #[ticket]</code>	Buscar en tu sistema de seguimiento y devolver solo enlaces. No se escribe ningún informe.
<code>Seed glossary: [text]</code>	Proponer términos de glosario a partir de un bloque de texto. Tú apruebas cada uno.
<code>Learn from my recent tickets</code>	Adoptar tu estilo a partir de los bugs que ya están en tu sistema de seguimiento.
Configuración y estado	
<code>Begin setup</code>	Configuración inicial completa, o cambiar una sola opción más adelante.
<code>Check status</code>	¿Están conectadas mis herramientas?

DI ESTO	QUÉ OCURRE
Check readiness	¿Qué falta antes de poder registrar?
Add a custom MCP server	Conectar una herramienta que no viene incluida.
Fix servers	Diagnosticar y reparar las conexiones MCP.
Check saved credentials	Solo nombres. BugIt nunca muestra el valor de una credencial guardada.
Talk to me in [language]	Cambiar el idioma en el que BugIt conversa contigo en esta máquina. Es distinto del idioma en que se escriben tus informes.
help	La versión corta de esta tabla, en el chat.
Licencia, actualizaciones y copias	
Activate	Autorizar esta máquina en tu navegador. Nunca se pega una clave ni una contraseña en el editor.
License status	Tu plan, tu titularidad y tu estado sin conexión.
What version am I on?	Tu versión, y si hay una actualización esperando.
Update	Instalar la última versión firmada en el mismo sitio. Tus ajustes se conservan y se copian antes.
Back up my settings	Guardar localmente la configuración, el glosario, el estilo y la configuración MCP. Las credenciales no se incluyen.
List backups	Ver las copias a las que puedes volver.
Restore my settings	Volver a una de ellas.
Create support bundle	Un archivo de diagnóstico depurado para adjuntar a un correo de soporte. Sin credenciales, sin tickets y sin rutas.
Equipo	
Authorize this Team device	Aprobar esta máquina con tu propia cuenta de BugIt, en el navegador.
Manage my devices	Abre el Portal de BugIt, donde se cierra la sesión de un dispositivo y se libera su puesto.
Manage my Team	Abre el Portal de BugIt, donde se gestionan personas, roles, puestos y licencias.

PARTE 3 · HAZLO TUYO: AÑADE EL CONOCIMIENTO DE TU PROYECTO

Recién instalado, Buglt es un agente de QA en blanco. Así es como le enseñas tu proyecto después de la compra. Sin programar, sobre todo hablando con él en el chat. Todo lo que añadas permanece en tu computadora.

1 · Las palabras de tu equipo (el glosario)

Esto hace que las traducciones y la predicción de categorías sean precisas para tu proyecto.

1. En la lista de archivos, abre `.github/glossary/terms.template.md`.
2. Completa la tabla con las palabras de tu equipo: nombres de funciones, nombres de pantallas, nombres de elementos, abreviaturas y (si usas dos idiomas) sus traducciones.
3. Guarda el archivo (`Ctrl+S`). Eso es todo; Buglt usa estos términos exactos a partir de ahora.

Atajo

Activa "glossary auto-seed" durante `Begin setup` y Buglt sugiere términos a partir de tus tickets existentes, y tú solo apruebas cada uno.

2 · Tu estilo de bugs (estilo propio)

¿Quieres que los tickets coincidan exactamente con el formato de tu equipo: estilo de título, etiquetas de severidad, campos obligatorios? No lo describes; lo **muestras**:

1. En el chat, di: "que coincida con el estilo de mi equipo."
2. Pega **una captura de pantalla** de un ticket existente de tu sistema de seguimiento.
3. El modelo de IA que ya está en tu chat lee la imagen conforme a los términos de su propio proveedor, Buglt elimina los datos personales del texto que extrae y guarda su formato, luego lo sigue en cada ticket futuro.

3 · Tus categorías, plataformas y campos

Solo dile a Buglt en lenguaje sencillo y actualizará tus ajustes por ti:

ESCRIBES (EJEMPLOS)

Mis categorías son Jugabilidad, UI, Audio y Redes
Probamos en Windows y Xbox

4 · Tus propias herramientas (conexiones personalizadas)

¿Usas una herramienta que no está en la lista integrada? Conéctala igual que una integrada:

ESCRIBES

Añade un servidor MCP personalizado

Asígnale un nombre corto y su dirección (debe ser `https://`); Buglt se conecta y comprueba su estado por ti.

5 · Simplemente corrígelo sobre la marcha

La forma más fácil de todas: cuando Buglt redacte un ticket, corrige lo que no te guste: una etiqueta, la redacción, la severidad. Con "learn from your edits" activado (recomendado), recuerda tu preferencia y la aplica la próxima vez. El conocimiento de tu proyecto se acumula de forma natural en tu máquina, y nada de eso se envía nunca a Taskivator.

Todo lo que añades sigue siendo tuyo

Tu glosario, tu estilo propio, tus categorías y tus preferencias aprendidas se guardan en tu computadora y se respaldan antes de cada actualización. El contexto relevante puede enviarse al modelo de IA y a las herramientas que conectes cuando Buglt realiza su solicitud. Nada de ello se envía a Taskivator, y un paquete para compartir sale de tu equipo solo cuando decides enviarlo.

6 · Comparte tu configuración con tu equipo (licencia Equipo)

Una vez que hayas configurado tu glosario, tu estilo propio y tus elecciones de sistema de seguimiento como quieres, comparte esa configuración con el resto de tu equipo, en lugar de repetir los pasos 1-3 en cada máquina:

EJECUTAS (UNA VEZ, DESPUÉS DE QUE TU CONFIGURACIÓN ESTÉ COMO QUIERES)

```
python tools/share.py export
```

Eso escribe `config.share.zip`: tus elecciones de sistema de seguimiento/fallos/comunicación, tu glosario y tu estilo propio, agrupados sin contraseñas ni tokens dentro. Envíaselo a tu equipo.

CADA COMPAÑERO EJECUTA PRIMERO, PARA REVISAR

```
python tools/share.py import config.share.zip
```

Esto muestra qué cambiaría y no escribe nada.

TRAS REVISARLO, PARA APLICAR LA CONFIGURACIÓN

```
python tools/share.py import config.share.zip --apply
```

Esto aplica tu configuración de sistema de seguimiento, fallos y comunicación, sin volver a escribir categorías. Para adoptar también tu glosario y tu estilo propio, deben revisarlos y elegir en su lugar `python tools/share.py import config.share.zip --apply --apply-instructions`. Su propia licencia y los ajustes de su dispositivo no se tocan, y no se transfiere ninguna credencial en el paquete.

Si la importación cambia hacia dónde se envían los datos

Si una configuración compartida añadiría una nueva conexión personalizada o dirección de notificación, la vista previa muestra exactamente qué cambiaría antes de aplicar nada. Al aplicar, también se toma primero una instantánea de la configuración actual del compañero, así que `Restore my settings` lo deshace si no era lo que esperaba.

PARTE 4 · ACTUALIZACIONES, COPIAS DE SEGURIDAD Y SEGURIDAD

Obtener actualizaciones (un solo comando)

Cuando hay una versión más reciente disponible, Buglt la menciona una vez cuando dices `hi`. Las actualizaciones de software están incluidas mientras tu licencia de Buglt esté activa. Para instalarla:

ESCRIBES

Update

1. Buglt hace primero una copia de seguridad reciente de tus archivos.
2. Descarga la nueva versión y comprueba que es genuina y no manipulada (está firmada criptográficamente, así que se rechaza una actualización falsa o alterada).
3. La instala sin tocar tus ajustes, tu glosario, tu estilo propio, tu licencia ni tus tokens.
4. Te pide que recargues VS Code (`Ctrl+Shift+P` → escribe "Reload Window" → Enter). Inicia un chat nuevo y estarás en la nueva versión.

Nunca se elimina ninguna carpeta

A diferencia de la primera instalación, las actualizaciones son in situ. Nunca eliminas ni vuelves a descargar nada, y tu configuración siempre se conserva y se respalda.

Qué es seguro editar a mano y qué no

Seguro, y se conserva en cada actualización: `config.json`, `.github/instructions/house-style.instructions.md` (consulta la Parte 3; es la forma admitida de personalizar el comportamiento), `.github/glossary/terms.template.md`, `.github/instructions/learned.instructions.md`, y `.vscode/mcp.json`.

No es seguro: no edites esto a mano

El archivo del agente (`.github/agents/bugit-qa-agent.agent.md`), cualquier otro archivo bajo `.github/instructions/`, y todo lo que haya en `tools/` son el producto en sí, no tu configuración. Una actualización los **sobrescribe en silencio**, y, a diferencia de los archivos anteriores, **no hay copia de seguridad desde la que restaurarlos**. Buglt comprueba esto al iniciar y de nuevo justo antes de instalar una actualización, y te avisa si encuentra algún cambio, pero la regla más segura es simplemente no editarlos.

Copias de seguridad y restauración

ESCRIBES	QUÉ HACE
<code>Back up my settings</code>	Guarda una instantánea local de tu config, glosario, estilo propio, endpoints MCP y aprendizaje. Tu titularidad de dispositivo firmada y los valores de credenciales se excluyen.
<code>List backups</code>	Muestra todas las instantáneas guardadas con su fecha.

`Restore my settings`

Retrocede a una instantánea (y antes toma una instantánea de tu estado actual, para que la propia restauración se pueda deshacer).

Automático antes de cada actualización

BugIt siempre hace una copia de seguridad antes de instalar una versión nueva, así que una actualización nunca puede perder tu configuración. Si tu configuración se ve mal después de una actualización, `Restore my settings` la soluciona.

Seguridad: qué está protegido

✓ Nada sin tu confirmación escrita

Cada ticket y comentario se muestra en vista previa; las acciones irreversibles requieren escribir `FILE IT`, y un simple «sí» no basta.

🧪 Dry-run = cero escrituras

Di `dry run` y tu sistema de seguimiento no se contacta en absoluto, incluidas las lecturas. El dry-run suprime todo ticket, comentario, edición, adjunto, eliminación y notificación externos (los comandos locales que ejecutas a propósito, como copia de seguridad o exportación, todavía pueden crear archivos locales) y tampoco hay búsqueda, ni comprobación de duplicados, ni comprobación de conexión, porque cada una de ellas también llega a tu tablero y desbloquea tu token guardado. `QA_AGENT_DRY_RUN=1` en tu terminal es el interruptor que los comandos incluidos aplican en el código; decir «dry run» en el chat pide al asistente que se detenga, lo cual es útil pero no es la misma garantía.

🔑 Sin secretos en archivos

Los tokens viven en el almacén de tu sistema operativo, nunca en un archivo de texto plano, nunca se envían a nosotros.

🔒 Se ejecuta en tu máquina

Tu trabajo solo llega a las herramientas que conectas y a tu propio modelo de IA.

Tus datos y privacidad

Lo único que BugIt envía a Taskivator son datos de licencia/actualización: el inicio de sesión en tu cuenta de BugIt (en el navegador, en el Portal), un identificador de dispositivo con hash unidireccional, un identificador de instalación aleatorio, la etiqueta de tu dispositivo (nombre de host) y el nombre de tu sistema operativo, la versión de BugIt y el filtro de plan que elijas, material criptográfico de desafío de vida corta, el token de aprobación de un solo uso del enlace del Portal que abrió tu navegador, un secreto de confirmación por activación (su hash, y el valor en sí solo si revocas el acceso de este dispositivo), y la titularidad Solo o Team que apruebas para este dispositivo. No hay ninguna clave de licencia, y tu contraseña permanece en el navegador y nunca se introduce en VS Code. Tus reportes de bug, especificaciones, glosario, ajustes y tokens nunca se envían a Taskivator. Lo que registras sí va adonde lo

envías: a tu propio sistema de seguimiento, en tu propia cuenta, con tu propia credencial. Si usas un modelo de IA alojado, tu borrador también llega a ese proveedor. Sin telemetría, sin analíticas, nunca. Los detalles completos están en el archivo [PRIVACY.md](#) de tu carpeta Buglt.

Riesgos y advertencias importantes: por favor, léelo

La IA puede equivocarse

Buglt redacta reportes con un modelo de IA, que puede malinterpretar detalles o inventar algo. Lee siempre la vista previa antes de confirmar por escrito ([FILE IT](#) para comentar; un simple «sí» no basta). Estás aprobando lo que se va a registrar.

Registra en tu sistema de seguimiento real

Una vez que apruebas, se crea un ticket real. ¿Estás practicando? Usa [dry run](#) para escribir el reporte sin registrarlo.

La seguridad de tu proyecto es responsabilidad tuya

Cómo usas Buglt, y la seguridad de tus proyectos, datos y sistema de seguimientos, son tu responsabilidad. Las salvaguardas reducen el riesgo pero no sustituyen tu revisión.

Lo que Buglt no hace

Buglt registra tickets en los once sistemas que nombra, cada uno con una credencial que creas en tu propia cuenta y que Buglt comprueba en el destino elegido antes de guardarla. Lo que llega a un campo de prioridad ordenable difiere según el sistema, y GitHub, GitLab y Trello no tienen ninguno, así que allí la gravedad se escribe en el texto del ticket. Confluence, Sentry y Notion son de solo lectura; un servidor MCP compatible, que configuras con ayuda de Buglt, solo hace falta para eso. Las notificaciones de Slack, Teams, Discord y correo son integradas. Usa tu IA (Copilot o tu propia clave de OpenAI/Anthropic), no un modelo incluido; la censura de datos es en la medida de lo posible; y funciona en VS Code, en la extensión de Claude y en un terminal normal. Una licencia = un dispositivo (Solo) o 5 puestos de miembro con un dispositivo cada uno (Team); un Equipo puede tener más de una licencia de Equipo, y sus puestos se suman. Los resultados y la consistencia dependen también de qué modelo de IA responda: uno más potente sigue los pasos de seguridad con más fiabilidad que uno muy ligero o gratuito.

PARTE 5 · SOLUCIÓN DE PROBLEMAS Y PREGUNTAS FRECUENTES

Cuando algo falle, descríbeselo primero al asistente en lenguaje sencillo. La mayoría de los problemas de configuración y de uso diario se pueden diagnosticar directamente en el editor.

✦ Prueba esto primero: pídele a la IA que lo arregle

Cuando algo falle (durante la configuración o el uso diario), tu solución más rápida es casi siempre **preguntarle al asistente directamente en el chat**. Describe lo que pasó con tus propias palabras (por ejemplo, "la configuración se detuvo a la mitad" o "mi sistema de seguimiento no se conecta") y pídele que lo arregle. La IA puede diagnosticar y reparar la mayoría de los problemas al instante. Por favor, prueba esto antes de escribirnos; resuelve la gran mayoría de los problemas en segundos.

Aplicar una corrección de la IA: el botón "Keep"

Si el asistente corrige algo cambiando un archivo, VS Code muestra una pequeña barra Keep / Undo. Si pediste la corrección y estás conforme con ella, haz clic en **Keep**, y el cambio se aplica solo a tu propia copia, en tu computadora. Haz clic en **Undo** para descartarlo. Nada de lo que mantengas con Keep se comparte con nadie más.

QUÉ VES	QUÉ HACER
"Activate to start"	Escribe <code>Activate</code> . BugIt abre el BugIt Portal en tu navegador. Inicia sesión, elige tu titularidad Solo o Team y aprueba este dispositivo. ¿Aún no tienes cuenta? Consigue una en la tienda.
El navegador no se abrió, o la activación no se completó	Escribe <code>Activate</code> de nuevo para reabrir el BugIt Portal, luego termina de iniciar sesión y de aprobar este dispositivo. Deja el comando en marcha hasta que diga que la autorización está guardada en este ordenador. Tampoco hay que pegar ninguna clave.
"No tracker enabled"	Escribe <code>Begin setup</code> y elige tu sistema de seguimiento.
Se rechaza el registro	Ejecuta <code>python tools/connect.py jira --check</code> . Te dice quién eres para el sistema de seguimiento, qué puede crear BugIt, y enumera en <code>fixes</code> el remedio concreto.
Sigue pidiéndote que actives	Escribe <code>Activate</code> y completa el inicio de sesión en el navegador. BugIt nunca muestra los valores de las credenciales guardadas. (¿Primera vez? Consulta el Paso 8c.)
No registra un bug	Escribe <code>Check readiness</code> ; enumera exactamente lo que falta (normalmente que todavía no has guardado el token de tu sistema de seguimiento).
Algo parece roto después de una actualización	Si cambió tu configuración, escribe <code>Restore my settings</code> . Si los archivos propios de BugIt están dañados, escribe <code>Repair my BugIt install</code> y revisa antes la vista previa. Si los archivos están bien y la nueva versión sigue fallando, escribe a <code>support@bugit.dev</code> .
El agente parece "salirse del guion"	Escribe <code>Read and follow all your given instructions</code> , o inicia un chat nuevo. Esto puede pasar con más frecuencia en un modelo de IA ligero/gratuito; un modelo más potente en el selector de modelos de Copilot es más consistente.
Las respuestas se sienten genéricas / no específicas del proyecto	Dale más contexto, o muéstrale un ticket existente durante <code>Begin setup</code> para que aprenda tu estilo. Vuelve a ejecutar <code>Begin setup</code> para refinarlo.

QUÉ VES

QUÉ HACER

Quieres cambiar una elección de la configuración

Escribe `Begin setup` y di qué quieres cambiar (p. ej., "añade un sistema de seguimiento" o "empezar de nuevo") y reabre solo esa parte. Si la configuración simplemente se interrumpió a la mitad, un `Begin setup` simple continúa donde lo dejó en lugar de reiniciar.

Comprobaciones de estado integradas

`Check status` (¿están conectadas mis herramientas?) · `Check readiness` (¿qué falta antes de poder registrar?). Para una comprobación más profunda, abre Terminal → New Terminal y ejecuta `python tools/selftest.py`.

Preguntas frecuentes

¿Necesito saber programar?

No. Si sabes instalar una app y escribir una frase, puedes usar Buglt. Se encarga de las partes técnicas por ti.

¿Registraré bugs sin que yo lo sepa?

Nunca. Cada ticket y comentario se muestra en vista previa y necesita una confirmación escrita: `FILE IT` para comentar; un simple «sí» no basta. Usa `dry run` para practicar sin ninguna escritura.

¿Tengo que iniciar un servidor cada vez?

No. El registro usa el token de API que guardaste con `python tools/connect.py`, así que no hay nada que iniciar. Un servidor MCP solo hace falta para las integraciones opcionales de solo lectura, como Confluence, Sentry y Notion.

¿Puedo usarlo en dos computadoras?

Un dispositivo a la vez en Solo. En un Equipo, cada miembro tiene su propia cuenta y su propio dispositivo, y el número de puestos proviene de las licencias de Equipo que tenga el Equipo: cada licencia de Equipo añade 5. Pasar a un dispositivo nuevo no es problema: activa el nuevo, y el anterior deja de poder renovar de inmediato. Su plaza se libera en cuanto el dispositivo anterior se reconecta (y confirma que ha cedido el acceso) o su licencia sin conexión de 72 horas expira, así que no hay ningún periodo de espera fijo. Escribe `License status` para comprobarlo.

¿Qué pasa si el servidor de licencias está caído?

Sigues trabajando. Tras una activación en línea correcta, una licencia almacenada en caché te permite trabajar sin conexión hasta 72 horas, así que una interrupción del servidor de licencias (o simplemente un fin de semana sin internet) no te detiene dentro de esa ventana. Pasado ese plazo se requiere una verificación en línea.

¿Necesito GitHub Copilot?

Es la forma más fácil. Si no lo tienes, ejecuta `python tools/setup_wizard.py` en la Terminal para configurarlo, y Buglt puede ejecutarse de forma independiente con tu propia clave de OpenAI/Anthropic (`python standalone/run.py`).

¿Son privados mis datos?

Sí. Taskivator solo recibe datos de licencia/actualización: el inicio de sesión en tu cuenta de Buglt (en el navegador, en el Portal), un identificador de dispositivo con hash unidireccional, un identificador de instalación aleatorio, la etiqueta de tu dispositivo (nombre de host) y el nombre de tu sistema operativo, la versión de Buglt y el filtro de plan que elijas, material criptográfico de desafío de vida corta, el token de aprobación de un solo uso del enlace del Portal que abrió tu navegador, un secreto de confirmación por activación (su hash, y el valor en sí solo si revocas el acceso de este dispositivo), y la titularidad Solo o Team que apruebas para este dispositivo. El software Buglt no utiliza la medición de Google Ads ni envía telemetría del producto. No hay ninguna clave de licencia, y tu contraseña permanece en el navegador. Tus tickets solo van al modelo de IA y al sistema de seguimiento que conectes, nunca a nosotros.

¿Puedo editar los archivos de Buglt?

Está pensado para que personalices tus partes: tu glosario, tu estilo propio y tus ajustes (Parte 3). Solo no desactives la licencia/activación. Si encuentras un bug real en la herramienta misma, escribe a soporte para que se pueda corregir para todos en la próxima actualización.

¿Con qué sistemas de seguimiento funciona?

Con once, y Buglt registra tickets en todos ellos: Jira Cloud, Azure DevOps, GitHub Issues, GitLab Issues, Bugzilla, YouTrack, Linear, Shortcut, ClickUp, Asana y Trello. Cada uno tiene configuración guiada, cada uno escribe con una credencial que creas en tu propia cuenta, y Buglt la comprueba en el destino elegido antes de guardarla. Lo que cambia es qué extras llegan: Buglt no sube archivos a GitHub, GitLab ni Bugzilla, y te lo dice antes de que lo intentes (GitHub no ofrece ninguna subida autenticada con credencial; GitLab y Bugzilla sí ofrecen una, que Buglt no ha certificado), ClickUp acepta un archivo y no puede quitarlo, y GitHub, GitLab y Trello no tienen campo de prioridad, así que allí tu gravedad se escribe en el texto del ticket. Elige el tuyo durante la configuración y cámbialo cuando quieras con [Begin setup](#).

¿Detectará bugs duplicados antes de que registre?

Sí, en un informe estándar. Antes de escribir nada, busca en tu sistema de seguimiento reportes parecidos (incluso los redactados de forma un poco distinta) y te muestra las coincidencias, para que puedas evitar crear el mismo bug dos veces. Tú sigues decidiendo si continuar. El formulario rápido omite esa búsqueda a propósito, para ahorrar el viaje de ida y vuelta, y su vista previa te lo indica.

¿Puede escribir tickets en más de un idioma?

Sí. Añade un segundo idioma durante la configuración y cada reporte se escribe en ambos, en bloques separados, usando tu glosario para que los términos del producto se mantengan exactos; nunca improvisa una traducción.

¿Puedo adjuntar una captura de pantalla?

Pide a Buglt que las adjunte. Crea una carpeta de evidencias, la abre y te indica la ruta exacta; deja ahí tus archivos y di que has terminado. Buglt revisa los archivos de texto en busca de credenciales y se detiene en lugar de adjuntar uno, y antes te muestra una vista previa. No puede ver dentro de una imagen: revisa tú mismo una captura de pantalla antes de aprobarla. Se adjuntan tras tu propio [FILE IT](#). Buglt no puede acceder a un archivo pegado en el chat, por eso siempre te ofrece la carpeta.

¿Oculta contraseñas y tokens en mis reportes?

Cuando la censura de datos está activada, elimina correos, tokens y direcciones IP de cada borrador antes de registrarlo. Siempre ves el texto completo primero en tu vista previa.

Registré un ticket por error. ¿Puedo deshacerlo?

La vista previa y la confirmación escrita ([FILE IT](#) ; un simple «sí» no basta) evitan la mayoría de los errores antes de escribir nada. Si activaste el registro de auditoría, escribe [Undo last filing](#) : muestra lo que Buglt escribió y puede retirar un comentario o adjunto que el propio Buglt añadió. Cerrar o borrar el ticket es siempre una acción tuya en tu sistema de seguimiento, porque ningún sistema le ofrece a Buglt una forma de hacerlo.

¿Puede ayudarme a verificar o cerrar un bug?

Sí. Pide un comentario de verificación y redactará una nota clara de aprobado/fallido (en tus idiomas) sobre el ticket existente, y tú la apruebas antes de que se publique.

¿Puedo usarlo para más de un proyecto?

Cada carpeta de Buglt está configurada para el sistema de seguimiento de un proyecto. Para otro proyecto, o vuelves a ejecutar [Begin setup](#) para apuntarlo a otro sitio, o mantienes una copia separada por proyecto.

¿Cómo obtengo actualizaciones?

Cuando sale una versión nueva, Buglt la menciona al iniciar un chat. Escribe [update](#) y se instala in situ: tu glosario, tu estilo propio y tus ajustes se conservan, y antes hace una copia de seguridad.

¿Qué pasa cuando expira mi licencia?

Cuando termina tu plazo, renueva desde tu cuenta; tu dispositivo aplica la titularidad renovada la próxima vez que se conecte en línea. El préstamo sin conexión de 72 horas mencionado arriba es independiente; solo cubre una interrupción temporal del servidor de licencias, no un plazo vencido. Escribe [License status](#) en cualquier momento.

Si renuevo o compro otra licencia, ¿se acumulan los días?

Los días nunca se acumulan. Una renovación **reemplaza** tu titularidad actual; tu plazo empieza de cero y los días no usados no se trasladan, así que renueva cerca del final de tu plazo. **Con los puestos es distinto.** En un Equipo, una segunda licencia de Equipo añade sus 5 puestos a los primeros, cada una por su propio plazo: dos licencias dan 10 puestos. Consulta las preguntas sobre Equipo más abajo.

Licencias de Equipo: preguntas

Todo lo de esta página ocurre en el **Portal de Buglt** en tu navegador, en la página **Equipo**, no en el editor. Si compraste una licencia Solo, puedes saltarte esta página.

¿Qué obtengo con una licencia de Equipo?

Cinco puestos durante un año, una licencia de 1 año sin suscripción que no se renueva sola. Cada persona del Equipo inicia sesión con su propia cuenta de Buglt y activa Buglt en un dispositivo propio: no hay clave compartida ni inicio de sesión compartido. Las personas, las invitaciones, los roles y los puestos se gestionan en la página Equipo del Portal.

¿Cómo añado a alguien?

Abre la página Equipo y envía una invitación a su dirección de correo; puedes pegar varias direcciones a la vez. Cada invitación solo sirve para la dirección a la que se envió, así que un enlace reenviado no le sirve a otra persona. Una invitación pendiente ya ocupa un puesto hasta que se acepta o se cancela, por eso un Equipo puede estar lleno antes de que todos se hayan unido.

¿Quién puede hacer qué?

La persona **propietaria** puede hacerlo todo, incluida la facturación y entregar el Equipo a otra persona. Un **administrador** puede invitar y quitar personas y cambiar los valores compartidos del Equipo. Un **miembro** usa Buglt y puede salir cuando quiera. La propiedad no se puede quitar: se transfiere.

Necesitamos más de cinco personas. ¿Puedo comprar otra licencia de Equipo?

Sí. Compra una segunda licencia de Equipo y añádela al mismo Equipo desde la página Equipo. **Los puestos se suman:** dos licencias dan 10 y tres dan 15. No hay un máximo fijo, porque cada puesto proviene de una licencia que posees. Añadirla surte efecto de inmediato.

Si añado una segunda licencia de Equipo, ¿cuándo expira mi Equipo?

Cada licencia conserva su propia fecha de fin y paga sus propios 5 puestos hasta entonces. Tu Equipo sigue funcionando mientras **alguna** de sus licencias esté vigente, así que la fecha de fin del Equipo es la más tardía de las dos. Cuando termina la primera, el Equipo pierde los 5 puestos que esa licencia pagaba; el Equipo en sí no se cierra. Una segunda licencia no alarga la primera: si a una le quedan tres meses y añades una licencia nueva de un año, tendrás 10 puestos durante tres meses y 5 durante los nueve siguientes. Renueva la licencia que termina antes para conservar los 10. La página Equipo muestra cada licencia con su propia fecha de fin, así que siempre ves cuál se acaba primero.

¿Qué pasa si acabamos con más personas que puestos?

Antes de que termine una licencia escribimos al Equipo tres veces: 30 días, 7 días y 1 día antes, así que nunca es una sorpresa. Si aun así termina mientras el Equipo ocupa más puestos de los que paga, primero las invitaciones pendientes ceden sus puestos y después pierden el acceso las personas que se unieron más recientemente. La persona propietaria nunca se retira, así que siempre hay alguien que puede renovar. No se borra nada de lo que se haya registrado; solo termina el acceso a Buglt.

¿Y si la persona propietaria de nuestro Equipo se va?

La persona propietaria puede nombrar un **sucesor** en la página Equipo. Si luego no está disponible, el sucesor pide hacerse cargo; se avisa a todo el Equipo y el cambio surte efecto 7 días después, salvo que la persona propietaria lo detenga antes. Quien sí esté disponible puede transferir el Equipo en cualquier momento, y para ello se le pide confirmar con autenticación de dos factores.

¿Puede trabajar igual todo el Equipo?

Sí. Configura Buglt como quieras, ejecuta `python tools/share.py export` y cada compañero ejecuta `import` sobre el archivo que le envíes (Parte 3, sección 6). La página Equipo además deja constancia del sistema de seguimiento, el proyecto, la persona asignada y las etiquetas que tu Equipo ha acordado, para que todos puedan consultarlo en un solo sitio. **Nunca pongas ahí un token ni una contraseña:** todo el Equipo puede leer esos valores. Las credenciales reales siempre se guardan por persona con `python tools/connect.py`, en su propia máquina.

Alguien ha dejado el Equipo. ¿Qué pasa con su puesto?

Quítalo en la página Equipo: su dispositivo se desconecta y el puesto queda libre enseguida para tu siguiente invitación. Un miembro también puede salir por su cuenta. En ambos casos su cuenta de Buglt sigue siendo suya; solo termina su acceso a tu Equipo.

¿Adónde van los recibos y los avisos de renovación?

A la dirección de la cuenta de la persona propietaria, de forma predeterminada. En la página Equipo puedes añadir un nombre de empresa y un correo de facturación aparte, y allí irán los avisos y recibos de ese Equipo. Eso no cambia quién es la persona propietaria.

PARTE 6 · LICENCIA Y SOPORTE

Términos de la licencia (resumen en lenguaje sencillo)

Los términos completos y vinculantes están en el archivo [LICENSE](#) de tu carpeta BugIt, y ese documento es el que rige. En resumen:

TEMA	EN PALABRAS SENCILLAS
Con licencia, no vendido	Compras una licencia para usar BugIt, no la propiedad. Taskivator conserva todos los derechos.
Tus puestos	Solo = un dispositivo a la vez. Equipo = 5 puestos de miembro por licencia de Equipo, un dispositivo cada uno, y más de una licencia de Equipo puede financiar el mismo Equipo. Tu cuenta y tu titularidad son personales; no las compartas, revendas ni publiques.
Revocación	Un acceso compartido, reembolsado, con contracargo o usado indebidamente puede ser revocado.
Personaliza, pero...	Edita tu config, glosario y plantillas libremente, pero no desactives ni evites la licencia/activación.
Tu responsabilidad	Cómo usas BugIt y la seguridad de tu proyecto son responsabilidad tuya. Tú revisas y apruebas cada reporte antes de que se registre. Las salvaguardas son ayudas, no garantías.
"Tal cual", responsabilidad	Se proporciona tal cual, sin garantía. En la medida en que lo permita la ley, la responsabilidad se limita a lo que pagues, excluyendo daños indirectos o consecuentes.
Plazo y ley aplicable	Una licencia de un año que comienza el día que la compras (sin suscripción, sin renovación automática), regida por las leyes de Japón. Los reembolsos se gestionan a través de la tienda donde compraste.
Los días no se acumulan, los puestos sí	Una renovación reemplaza tu titularidad actual: tu plazo empieza de cero y los días no usados no se trasladan, así que renueva cerca del final de tu plazo. En un Equipo, otra licencia de Equipo añade sus puestos por su propio plazo; el Equipo sigue activo hasta que termina su última licencia vigente. Cuando termina un plazo, renueva desde tu cuenta y tu dispositivo se vuelve a autorizar en su próxima comprobación en línea.

Dos documentos en tu carpeta

[LICENSE](#) (términos completos) y [PRIVACY.md](#) (declaración de privacidad). Al activar y usar BugIt, aceptas la LICENSE.

Cómo obtener ayuda

Por favor, revisa primero la Solución de problemas y las Preguntas frecuentes de arriba; cubren casi todo. Luego, en este orden:

1 · Pídele a la IA que lo arregle

Tu mejor primer movimiento: describe el problema en el chat y pídele al asistente que lo arregle. Si cambia un archivo y se ve bien, haz clic en Keep para aplicarlo (solo en tu PC).

2 · Ejecuta una comprobación de estado

`Check status` · `Check readiness` · o `python tools/selftest.py` en la Terminal.

3 · Restaura

`Restore my settings` recupera tu configuración, tu glosario y tu estilo propio; no deshace una actualización. Archivos de BugIt dañados: `Repair my BugIt install`.

4 · Contacta a una persona

Solo si la guía y la IA no pudieron ayudarte: visita bugit.dev o escribe a support@bugit.dev (el soporte se atiende en inglés). ¿Problemas de configuración en tus primeros 7 días? Te pondremos en marcha o te ayudaremos a conseguir un reembolso a través de la tienda.

Por favor, mantén la información confidencial de tu proyecto fuera de los correos de soporte

Cada proyecto es diferente, y buena parte de tu trabajo puede ser confidencial. Si nos escribes, por favor describe el problema de BugIt solo en términos generales, y no pegues código confidencial, detalles de bugs, especificaciones, capturas de pantalla ni datos de tu proyecto. Taskivator no se hace responsable de ninguna información confidencial que nos envíes, y cualquier contenido confidencial que recibamos se elimina de inmediato. Siempre que sea posible, pídele ayuda primero al asistente de IA; puede resolver la mayoría de los problemas justo dentro de tu editor.

Gracias por elegir BugIt.

Registra tickets limpios, evita los duplicados y recupera tu tiempo, un bug a la vez.

BugIt QA Agent · Guía del usuario · © 2026 Taskivator. All Rights Reserved. · bugit.dev · Los archivos LICENSE y PRIVACY.md incluidos con tu descarga son los documentos vinculantes.